

박수민_0617_Circular queue & pc command processing

```
#main.c
/*
 * 04.UART.c
 *
 * Created: 2026-06-16 오전 9:51:10
 * Author : kccistc
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h> // sei, ISR 등
#include <stdio.h>

#include "button.h"

extern void init_button(void);
extern void init_led();
extern int get_button(int button_num, int button_pin);

extern void init_uart0();
extern void UART0_transmit(uint8_t data);
extern void pc_command_processing();

FILE OUTPUT = FDEV_SETUP_STREAM(UART0_transmit, NULL, _FDEV_SETUP_WRITE);

void init_timer0();
volatile uint32_t msec_count = 0; // volatile의 최적화 방지

ISR(TIMER0_OVF_vect)
{
    TCNT0 = 6; // TCNT0 6~256: 250 펄스 count 하기 위해
    msec_count++;
}

int main(void)
{
    init_led();
    init_timer0();
    init_uart0();
    stdout = &OUTPUT; // printf가 동작할 수 있도록 stdout을 설정
```

```

sei(); // 전역(대문) interrupt 허용

init_button();

while (1)
{
    pc_command_processing();
}

void init_timer0()
{
    // TCNT0 = TimerCount0
    // TCNT0 6 ~ 256 : 250개 펄스를 count 하기 위해 6부터 시작
    TCNT0 = 6;

    TCCR0 &= ~( 1<< CS02 | 1 << CS01 | 1 << CS00); // 0분주 reset 먼저
    TCCR0 |= 1 << CS02 | 0 << CS01 | 0 << CS00; // 64분주
    TIMSK |= 1 << TOIE0; // TIMER0 Overflow INT
}

#include <uart0.h>
...

/*
 * uart0.h
 *
 * Created: 2026-06-16 오전 9:58:08
 * Author: kccistc
 */

#ifndef UART0_H_
#define UART0_H_
#define F_CPU 16000000UL // 16MHz
#include <avr/io.h> // PORTA PORTB PORTD... IO관련 reg가 들어 있다.
#include <util/delay.h> // _delay_ms _delay_us 등
#include <avr/interrupt.h>
#include <string.h>

```

```
#define QUEUE_SIZE 10
#define QUEUE_LENGTH 80
volatile int rear=0; // interrupt에서 queue에 data를 저장하는 위치값
volatile int front=0; // pc_command_processing에서 가져가는 위치값
volatile unsigned char rx_buff[QUEUE_SIZE][QUEUE_LENGTH];
```

```
#endif /* UART0_H_ */
...
```

```
#uart0.c
...
```

```
/*
 * uart0.c
 *
 * Created: 2026-06-16 오전 9:57:55
 * Author: kccistc
 */
```

```
#include "uart0.h"
#include <stdio.h>
void init_uart0();
void UART0_transmit(uint8_t data);
void pc_command_processing();
```

```
extern int func_state;
extern void (*fp[]) ();
```

```
// p.278 표 12-3
// pc로부터 1byte 들어오면 자동으로 이곳에 진입
// ex) led_all_on\n 이면 11번 이곳으로 진입
```

```
ISR(USART0_RX_vect)
{
```

```
    // interrupt 쓰는 변수는 volatile로 해야함
    volatile uint8_t data;
    volatile static int i=0;
```

```
    // UDR0를 data에 복사하는 순간 UDR0의 내용은 없어지므로, 데이터가 '이동'한다고
    생각해도 무방함
    data = UDR0;
```

```

    if (data == '\n' || data == '\r')
    {
        if ((rear+1) % QUEUE_SIZE == front % QUEUE_SIZE)
        {
            return; // queue full 상태
        }
        rx_buff[rear][i] = '\0';          // 문장의 끝인 NULL을 넣는다.
        i = 0;                             // 다음 string
을 저장하기 위해 i를 0으로 만든다.
        rear = (rear + 1) % QUEUE_SIZE; // 0~9
    }
    else
    {
        if ((rear+1) % QUEUE_SIZE == front % QUEUE_SIZE)
        {
            return; // queue full 상태
        }
        rx_buff[rear][i++] = data;
    }
}

/*
1. 전송속도 : 9600bps
2. start/stop 설정
3. RX(수신): interrupt 로 설정

*/
void init_uart0(void){
    UBRR0H = 0x00;
    UBRR0L = 207;          // 9600bps, 표 8-9
    UCSR0A |= 1 << U2X0;  // 2배속 설정
    // UART0 송신, 수신이 모두 가능하도록 RX Interrupt가 가능하도록 설정.
    UCSR0B |= 1 << RXEN0 | 1 << TXEN0 | 1 << RXCIE0; // RXCIE: RX complete
interrupt enable
}

// UART0로 1byte 전송하는 함수
void UART0_transmit(uint8_t data)
{
    while( !(UCSR0A & (1 << UDRE0))); // no operation -> data가 송신중이면 송신이
끝날 때까지 기다림

```

```

        UDR0 = data;                // HW 전송 register에 data를 송신한다.

    }

void pc_command_processing()
{
    if (front != rear)                // data가 rx_buff에 존재하는지 check
    {

        printf("%s", rx_buff[front]);    // printf("%s", &rx_buff[front][0]
        if (strncmp((const char *)rx_buff[front], "led_shift_left_on",
strlen("led_shift_left_on"))==0) {
            func_state = 0;
        }
        else if (strncmp((const char *)rx_buff[front], "led_shift_right_on",
strlen("led_shift_right_on"))==0) {
            func_state = 1;
        }
        else if (strncmp((const char *)rx_buff[front], "led_shift_left_keepon",
strlen("led_shift_left_keepon"))==0) {
            func_state = 2;
        }
        else if (strncmp((const char *)rx_buff[front], "led_shift_right_keepon",
strlen("led_shift_right_keepon"))==0) {
            func_state = 3;
        }
        else if (strncmp((const char *)rx_buff[front], "led_flower_on",
strlen("led_flower_on"))==0) {
            func_state = 4;
        }
        else if (strncmp((const char *)rx_buff[front], "led_flower_off",
strlen("led_flower_off"))==0) {
            func_state = 5;
        }
        front = (front+1) % QUEUE_SIZE;
    }

    fp[func_state] ();

};
...

```

```

#led.c
/*
 * led.c
 *
 * Created: 2026-06-15 오후 1:17:24
 * Author: kccistc
 */

#include "led.h"
void init_led(void);
void led_all_on(void);
void led_all_off(void);
void led_shift_left_on(void);
void led_shift_right_on(void);
void led_shift_left_keepon(void);
void led_shift_right_keepon(void);
void led_flower_on(void);
void led_flower_off(void);

extern volatile uint32_t msec_count;
#define FUNC_SU 6

int led_main(void);

int func_state = 0;
void (*fp[]) () =
{
    led_shift_left_on,      // func_state = 0
    led_shift_right_on,
    led_shift_left_keepon,
    led_shift_right_keepon,
    led_flower_on,
    led_flower_off         // func_state = 5
};

int led_main(void)
{
    // uint8_t led_toggle=0;
    init_led();
    led_all_off();

```

```

while(1)
{
    #if 1
    fp[func_state] ();

    #else
    //if (msec_count >= 500) // 500ms if (msec_count == 500)
    //{
        //msec_count = 0;
        //led_toggle = !led_toggle;
        //if (led_toggle) {
            //led_all_on();
        //}
        //else led_all_off();
    //}
    #endif
    // led_flower_on();
    // led_flower_off();
    //led_shift_left_keepon();
    //led_all_off();
    //led_shift_right_keepon();
    //led_all_off();
}

return 0;
}

void init_led(void)
{
    DDRA=0xff; // PORTA 를 출력 모드로 설정
    PORTA=0x00; // PORTA에 물려있는 led를 all off
}

void led_all_on(void)
{
    PORTA=0xff;
}

void led_all_off(void)
{
    PORTA=0x00;
}

```

```

void led_shift_left_on()
{
    #if 1
        static int i = 0;
        if (msec_count >= 100)
        {
            msec_count = 0;
            *((unsigned char *) 0x3B = 1 << i;
            i = (i+1) % 8;
            //if ((i = (i + 1) % 8) == 0) // 다음 index 값 계산
            //{
                //func_state = (func_state + 1) % FUNC_SU; // 다음 실행할
func 으로 jump
                //}
            }

        #endif

        #if 0 // original

            static int i = 0;
            *((unsigned char *) 0x3B = 1 << i;
            _delay_ms(30);

            i = (i + 1) % 8; // 다음 index 값

        #endif
    }

void led_shift_right_on()
{
    #if 1
        static int i = 0;
        if (msec_count >= 100)
        {
            msec_count = 0;
            *((unsigned char *) 0x3B = 0x80 >> i;
            i = (i+1) % 8;
            //if ((i = (i + 1) % 8) == 0) // 다음 index 값 계산
            //{
                //func_state = (func_state + 1) % FUNC_SU; // 다음 실행할
func 으로 jump
                //}
    }

```



```

    }

    #else
    for (int i=7; i>=0; i--)
    {
        // PORTA = 1 << i;
        *((unsigned char *) 0x3B) = 1 << i;
    }
    #endif
}

void led_shift_left_keepon()
{
    #if 1
        static int i = 0;
        if (msec_count >= 100)
        {
            msec_count = 0;
            *((unsigned char *) 0x3B) |= 1 << i;
            i = (i+1) % 8;
            //if ((i = (i + 1) % 8) == 0) // 다음 index 값 계산
            //{
                //led_all_off();
                //func_state = (func_state + 1) % FUNC_SU; // 다음 실행할
func 으로 jump
                //}

            }

        #else
        for (int i=0; i<8; i++)
        {
            // PORTA |= 1 << i;
            *((unsigned char *) 0x3B) |= 1 << i;

        }

        #endif
    }

    void led_shift_right_keepon()
    {
        #if 1
            static int i = 0;

```

```

    if (msec_count >= 100)
    {
        msec_count = 0;
        *(unsigned char *) 0x3B |= 0x80 >> i;
        i = (i+1) % 8;
        //if ((i = (i + 1) % 8) == 0) // 다음 index 값 계산
        //{
            //led_all_off();
            //func_state = (func_state + 1) % FUNC_SU; // 다음 실행할
func 으로 jump
            //}
        }

    #else
        for (int i=7; i>=0; i--)
        {
            PORTA |= 1 << i;
        }
    #endif
}

void led_flower_on()
{
    static int i = 0;
    if (msec_count >= 100)
    {
        msec_count = 0;
        *(unsigned char *) 0x3B |= (1 << (4 + i)) | (1 << (3 - i));
        i = (i+1) % 8;
        //if ((i = (i + 1) % 8) == 0) // 다음 index 값 계산
        //{
            //func_state = (func_state + 1) % FUNC_SU; // 다음 실행할
func 으로 jump
            //}
        }
    }

void led_flower_off()
{
    static int i = 0;
    if (msec_count >= 100)
    {

```

```
msec_count = 0;
*(unsigned char *) 0x3B &= ~((1 << (7 - i)) | (1 << i));
i = (i+1) % 8;
//if ((i = (i + 1) % 8) == 0) // 다음 index 값 계산
//{
//func_state = (func_state + 1) % FUNC_SU; // 다음 실행할
func 으로 jump
//}
}
}
```